HacdsDeFi Whitepaper

2026-11-11　Third Edition (contract #3000)

> This document describes the Hacash mainnet contract `HacdsDeFi3000.fitsh` (contract address `T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`, asset serial 3000).
> Every number here is a constant in the contract source. None of them can be changed after deployment.

Before We Start

hacds is an on-chain asset on the Hacash mainnet: serial 3000, decimal 0.

It runs on Hacash's first DeFi contract. Hacash's HVM (the Turing-complete smart contract VM) activated at block 765,432. This contract is the first implementation on mainnet to put all four DeFi primitives — AMM market making, lending, liquidation, and a treasury — into a single contract.

The difference between this edition and the previous one is not parameter tuning.

It is that **three centralized components were deleted outright**:

Previous edition	This edition
Off-chain oracle scores diamonds, signs a certificate; the contract verifies the signature	The contract reads diamond genes on-chain and scores them itself
Six admin parameter functions	No admin. Not one.
`set_paused` global kill switch	No pause switch
serial 2000, decimal 8	serial 3000, decimal 0
One diamond per deposit	Up to 200 diamonds per deposit

What was deleted does not come back. This is a design choice, not a roadmap.

This document has six parts: what went wrong last time, the design of this edition, four defining characteristics, the system architecture, current status, and known limitations.

---

1. The Previous Edition

1.1 The original design

The previous edition defined hacds as an inscription carved into HACD diamonds. The design had four parts.

Issuance by inscription. Each HACD carries one Hacds inscription; only one per diamond is valid. An ordinary diamond paid 10,000 hacds. How much a special diamond paid (500,000 for a heart, 300,000 for four consecutive letters, 200,000 for an unusual shape) was decided by community vote.

Trading through a plugin. The decentralized exchange was a plugin wallet called MoneyNex. Users installed the plugin and used its wallet to get in and out.

Liquidity through exchanges. The whitepaper's first listing target was xeggex.com.

Discovery by chain scanning. Every client scanned the whole chain for inscriptions. Third-party servers could offer search on top of that.

1.2 We built it, and we listed it

These were not plans on paper. The inscription system shipped, the MoneyNex plugin ran, and hacds listed on two exchanges: xeggex.com and nonkyc.io. Then trading was suspended.

1.3 What went wrong

The root cause is the first item. The other three are its consequences.

Price had no anchor. The issuance ratio was set by vote, which meant how many hacds each class of diamond was worth could be changed at any time by a vote. Exchanges had no reproducible conversion basis, market makers could not quote stably, and depth never developed.

Trading depended on someone else. The decentralized exchange was a third-party plugin. The plugin stops being maintained, the exchange stops. The centralized exchanges were even more direct: trading and circulation were in the hands of xeggex and nonkyc.

There was no fallback. Those two exchanges were the entire circulation channel. If they stopped, there was nowhere else on-chain to trade.

Put together: if the token cannot circulate freely, it cannot be priced; if it cannot be priced, no one can make a market; with no market, it depends entirely on exchanges; when the exchange stops, everything stops.

Listing was never the point. Even once listed, the design flaws did not

disappear — they were merely postponed until the day trading was suspended. That is the direct reason this edition changed the design.

1.4 What this edition changed

One thing first: the token was taken off the diamonds.

hacds became a standard on-chain asset, not attached to any diamond, freely divisible and transferable. The financial logic moved from plugins and exchanges into the contract. Circulation no longer depends on listings; the on-chain AMM trades by itself. A listing is now optional rather than a precondition for survival.

On top of that, the two remaining centralized components from the previous edition — **the oracle** and **the admin** — were also deleted.

---

2. The Design of This Edition

2.1 Where the supply comes from

The total supply of hacds is not arbitrary. It is computed from all diamonds on the network.

Each HACD is scored on five HIP-5 dimensions (shape, pattern, color, naming, number). The contract sets `SCORE_SCALE = 1e8`: **one point mints 100 million hacds**. The total uses the exact network-wide mean:

```
Total network score = mean score × total HACD issuance
                    = 3.55440306 × 16,777,216
                    = 59,632,987 points

Total supply        = total score × 1e8 per point
                    = 59,632,987 × 100,000,000
                    = 5,963,298,700,000,000 hacds
```

Anyone can recompute this: pull the explorer API, score every diamond with the same algorithm, and arrive at the same number. This is exactly what the previous edition lacked — a baseline that does not depend on anyone's decision.

2.2 How scoring works: read on-chain, no oracle

The previous edition scored off-chain, signed a certificate, had the contract verify the signature, and recomputed on-chain to compare. **This edition deletes that entire path.**

The contract now asks the VM directly for diamond genes within the same

transaction that receives the HACD:

```
hacd_number_batch(names)        → each diamond's number (u64)
hacd_visual_gene_batch(names)   → each diamond's visual gene (10 bytes)
```

With the genes in hand, the contract scores them itself across the five HIP-5 dimensions. **No off-chain involvement, no certificate, no signature, and nothing that can be forged.**

This capability depends on the "HACD gene view" that Hacash nodes provide from v1.3.0 onward, activated at block height 800,000. Both conditions are required — an older node cannot supply the genes, and a lower height means the VM rejects the call.

2.3 Batching: 200 diamonds at once

This is the second change v1.3.0 brought.

The previous edition could deposit only one diamond per transaction. The reason was gas: scoring one diamond in FitSH burned about 3,270 units of the `resource` bucket, and that bucket's limit is 6,100. Two diamonds overflowed it.

This edition moves the scoring out of FitSH and into an **RV64IM calc kernel** (`score_sum`). A single call passes the names, numbers, and genes of up to 200 diamonds and returns the total score:

```
input   names[6n] ‖ numbers[8n] ‖ visual_genes[10n]
output  Σ score (8-byte big-endian u64)
```

Scoring 200 diamonds costs 3,587 units of `compute` — one bucket handles it.

200 diamonds, one transaction, one signature.

Deposits batch; redemptions remain one diamond at a time. The reason is in §6.1.

2.4 Asset specification

Item	Value
serial	3000
ticket / name	HACDS / hacds
decimal	**0** (1 smallest unit = 1 hacds)
Total supply	5,963,298,700,000,000
issuer	T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH (the contract itself)
Issuance cost	8 HAC (protocol fee, floats with block reward)

The on-chain asset at #1028 also uses the ticket HACDS (its name is "HACD Fragments" — someone else's asset). The protocol does not check whether name and ticket collide; it only checks whether a serial is taken. **The protocol recognizes serial 3000.**

> ⚠ The old serial 2000 (address `VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1`) is retired. The two serials are independent assets: not interchangeable, not convertible.

---

3. Four Defining Characteristics

3.1 Open source

The contract (fitsh source plus the RV64IM kernel), the indexer, the transaction build service, and the wallets (PC and Android) are all open source.

A DeFi protocol runs on other people's money. The only way users can protect themselves is to read the code and check the math. The previous edition's wallet was a third-party plugin — users could only choose to trust it or not. This edition puts the logic in plain sight.

With the code public, three things become directly possible:

- **Scoring can be independently recomputed.** Anyone can pull every diamond on the network, rescore it with the same algorithm, and check whether issuance matches what the contract paid out. On testnet, 119 differential test cases confirmed the kernel matches the FitSH reference implementation exactly.
- **Contract behavior can be audited.** Having no admin means the function list contains no entry point that can change a parameter, change a balance, or pause the protocol. This can be verified function by function — you do not have to take our word for it.
- **Services can be replaced.** The indexer and the transaction build service are stateless HTTP services. Anyone can run their own and point a client at it.

3.2 Decentralization

This needs boundaries. Saying "fully decentralized" without qualification means nothing.

What is done:

Rules and execution live on-chain. AMM pricing, lending eligibility, liquidation thresholds, and fee distribution are all contract code. Participants trade under the same rules. We are not in the middle and cannot insert ourselves.

There is no admin. The previous edition had six admin functions (`set_fee`,

`set_loan_rate`, `set_liquidate_bonus`, `set_paused`, `set_oracle`, `rent_batch`). This edition has **none of them**. Calling one on-chain returns `CallNotExist(51)` — not "you lack permission," but **the function does not exist**.

Every parameter is a compile-time constant. Fees, the rate curve, the liquidation line, the reserve target — all written into the source. After deployment no one can change them, including us.

The contract cannot be upgraded and has no back door. There is no self-destruct, no upgrade function, no per-user freeze function. What is deployed is what stays.

Participation is permissionless. Swapping, market making, lending, liquidating, and even key renewal (`rent_keys`) are open to anyone. No whitelist, no approval.

Assets stay in your hands. hacds is a standard on-chain asset held at the user's own address. We do not custody it and cannot freeze it.

What is not done, stated plainly:

The treasury key is a single point. Fees accumulate at the treasury address `1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`, a cold wallet whose private key only we hold. The contract has no withdrawal function, so no one can take user assets; but **the protocol revenue in the treasury** is controlled by that one key. This is the only place this edition asks you to trust the project team.

The servers are single points. The indexer (a borrower ledger) and the transaction build service run on our hardware. They **hold no private keys and do not participate in consensus**. Users can substitute their own, or sign offline and broadcast. But when they are down, convenience features in the web app are affected.

These two are the actual boundaries of decentralization here.

3.3 Security

Two halves: how your money is kept, and how the protocol's rules are guaranteed.

Private keys never touch the network. The wallet is split into cold and hot halves. The offline machine builds and signs, the online machine only queries and broadcasts. The private key appears exactly once in the main process's memory, passed to a child process via an environment variable, never written to disk.

This works because Hacash transactions have no nonce and no timestamp expiry — a transaction signed offline remains valid whenever it is eventually broadcast.

Signing happens locally. Transactions are signed by the bundled Rust program on the local machine, never through a node. Nodes are used only to translate, verify, and broadcast. The node's `/util/transaction/sign` endpoint requires sending the private key over HTTP; we do not use it.

Wallet files are encrypted. The password is stretched with scrypt ($N=2^{15}$, r=8, p=1) and encryption is AES-256-GCM with the format version bound as AAD. The PC and Android wallets share one keystore format (`hacds-keystore v1`) and can import each other's files.

The UI process has no system privileges. The renderer enables `contextIsolation` and disables `nodeIntegration`, and can only reach the main process through a preload whitelist. Even if the renderer were fed malicious content, it could not reach the filesystem or the private key.

Scoring cannot be forged. Scoring happens in the contract, on inputs returned by the VM. There is no off-chain step to compromise and no signature to forge.

Assets are conserved. The on-chain identity holds: what users hold plus what the contract holds equals total supply. Every operation preserves it, and anyone can check at any time.

3.4 DeFi completeness

The previous edition's "finance" was two things: issue a token, and list it on an exchange. No financial primitives — no market making, no collateral, no borrowing, with returns only from price appreciation.

This edition fills that gap. One contract, four modules:

Module	Content
AMM	Two-way HAC ↔ hacds. Constant product `x × y = k`, 0.5% fee, **LP 80% / treasury 20%**
Lending	Post hacds as collateral, borrow HAC. 200% borrow ratio, 160% liquidation line, floating 2%–46% rate
Liquidation	Two tiers: normal (105%–160%, half the debt at a time) and deep-water (<105%, full)
Treasury	Fees accumulate; anyone can trigger a sweep into the cold wallet

Plus several tools.

Atomic transactions. Multiple actions in one transaction, all or nothing. The protocol cap is 200 actions.

Socialized bad debt (hcut). In extreme cases bad debt is spread proportionally

across all depositors, expressed as a global factor `hcut` that only decreases. This is a public, one-time, irreversible loss — not a bug, but the last line of defense.

Storage rent. Hacash contract state pays rent, and keys expire. The contract offers permissionless `rent_keys`: anyone can renew and earn 0.07 HAC per key. This is how the keeper works, and anyone can run one.

Lazy interest accrual. No cron job, no keeper dependency. Interest is caught up by block-height difference whenever someone interacts with the contract. Saves gas and cannot stall just because nobody is maintaining it.

---

4. System Architecture

Four layers: on-chain contract and asset, the node and supporting services, client wallets, and the cold/hot separation.

4.1 Three layers and their responsibilities

Layer	Component	Can	Cannot
(1) On-chain	HacdsDeFi contract · `T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`	Move assets, record debt, liquidate, score — per code	Be modified or paused by any address
(1) On-chain	hacds asset · serial 3000 · decimal 0	Be split, transferred, listed	Be inflated (no mint function)
(2) Services	Full node RPC · `node.hacds.com`	Read chain, verify, broadcast	Obtain private keys (we do not use its signing endpoint)
(2) Services	API proxy · `api.hacds.com`	Forward to the node, add CORS + HTTPS	Change any semantics
(2) Services	Indexer · `indexer.hacds.com`	Scan the chain, record who has debt and who is liquidatable	Affect any on-chain state
(2) Services	Build service · `build.hacds.com`	Turn human input into an unsigned transaction	Sign (private keys stay on the client)
(3) Clients	PC console / Android	Sign locally, broadcast	Bypass contract rules

Data flow: clients read state through "API proxy → full node" and read the ledger from the indexer. To send a transaction, they can construct it themselves

or ask the build service for an unsigned transaction — **signing always happens locally on the client**.

⚠ **One hard constraint**: full-node sandbox calls are limited to 2 concurrent requests per IP. Excess requests are rejected outright, not queued. Clients must serialize with backoff.

Why service addresses are subdomains, not `IP:port`: Cloudflare Workers can only reach ports 80 and 443; any `IP:8082` address is rejected before a connection is attempted. Everything external therefore goes through `*.hacds.com` subdomains backed by a Tunnel.

4.1 Responsibility boundaries

Component	Can	Cannot
Contract	Move assets, record debt, liquidate per code	Be modified or paused by any address
Full node	Read chain, verify, broadcast	Obtain private keys (we do not use its signing endpoint)
Indexer	Record who has debt, compute health	Affect any on-chain state
Build service	Turn parameters into an unsigned tx	Sign (private keys stay on the client)
Client	Sign locally, broadcast	Bypass contract rules

---

5. Current Status

5.1 Completed

Item	Status
Contract source	Final. 16,198 bytes (limit 16,384). Full security audit passed.
Scoring kernel	RV64IM kernel, 2,026 bytes. 119 differential tests match the FitSH reference.
Testnet	Local full node plus multiple addresses. Groups A/B/C/G — 40 functional tests — all pass.
Attack tests	43 attack scenarios (reentrancy, front-running, price manipulation, underflow, etc.)
Services	Node, indexer, build service, keeper all online with autostart on boot
PC wallet	v0.3.1 — Windows installer plus portable build
Android	v0.2.0 — signed APK, arm64
Website	hacds.com (whitepaper, docs, web console)

5.2 Not yet done: deployment

The contract is not yet deployed to mainnet. There is exactly one reason: the VM upgrade activation height has not been reached.

Condition	Status
Node ≥ v1.3.0 (provides the HACD gene view)	✅ Ready
Block height ≥ 800,000	⏳ Currently 798,812; about 1,188 blocks remaining

Both are required. Deployment happens when the height is reached.

Until then, the website and wallets are built to #3000 specifications; the contract address they point at will have no code until deployment.

5.3 What testnet verified

- **Batch deposits**: 54 diamonds in a single transaction, confirmed; differential tests cover the full 200-diamond cap
- **AMM**: constant product exact to the unit; slippage protection and the first-deposit dead shares both behave correctly
- **Lending**: borrow limits, interest accrual, repayment, and health views all match expectations
- **Liquidation**: both normal and deep-water liquidation were genuinely triggered, with accounting closing exactly
- **Transfers**: HAC, hacds, and HACD all transferred between addresses
- **Attack simulation**: all 43 scenarios failed as expected; no exploitable path

Two real defects were found and fixed, both on the liquidation path and both exposed only by real on-chain execution on testnet: one `resource` bucket overflow and one underflow. The fix notes are in `hacds_exchange/修复说明-清算gas与下溢.md`.

---

6. Known Limitations

Some of these are not omissions but deliberate trade-offs. They are written down so nobody has to rediscover them.

6.1 Redemption is still one diamond at a time

Deposits batch to 200; redemptions are one at a time. This is intentional.

Redemption has to price the "cherry-picking premium": redeeming a high-scoring diamond costs more, at a rate depending on that diamond's score relative to the

pool average. In a batch every diamond is judged differently, and redemption also triggers automatic AMM top-up, so each one needs its own balance check. More fundamentally, **accounting semantics** get hard: if diamond number 47 fails mid-batch, how do you roll back the 46 already processed? The contract cannot be upgraded, and getting that rollback wrong means locked funds.
So: **batch deposits, single redemption.** The wallet automatically splits your selection into multiple transactions; the user clicks once.

6.2 The treasury is a single point

A cold wallet, fixed. It is the project's revenue account and the only place you have to trust us.

6.3 The servers are single points

The indexer and build service run on our hardware. They cannot affect on-chain state, but when they are down the web app is less convenient. Anyone can run their own — the source is public.

6.4 300-second blocks

A successful broadcast is not confirmation; `ret:0` only means it reached the mempool. The wallet re-polls on the block cadence (15, 45, 90, 180, 320 seconds).

6.5 Interest is not displayed separately

The contract accrues interest lazily, and the accrual start has no public getter. Current interest owed cannot be computed from outside; it shows up only as the settled result at repayment time.

6.6 Withdrawals can be temporarily refused

If too much has been lent out and cash is short, HAC depositors' withdrawals are refused. This is deliberate — the protocol would rather have you wait than ship a queue that could deadlock. Two layers of protection make it rare: total borrowing is capped at 50%, and only 30% of the AMM's HAC can be lent out.

6.7 The contract cannot be upgraded

A bug found later cannot be fixed. That is why the parameters are conservative (a 20% liquidation buffer, a 50% borrow cap, two layers of cash protection), and one more reason: **do not put your entire net worth in**.

7. FAQ

Will hacds inflate?

No. It is issued against diamond value with a fixed total of 5,963,298,700,000,000 and no mint function. On deposit the contract checks `already issued + this mint ≤ total supply` and rejects anything over it (return 53).

Is there an admin who can take my money?

There is no admin. The contract contains no `set_*` function at all; calling one returns `CallNotExist(51)` — the function truly does not exist, it is not a permission failure. No one can pause it, change parameters, or move user assets.

How many diamonds can I deposit at once?

1. One transaction, one signature.

Why only one diamond per redemption?

See §6.1. The cherry-picking premium must be priced per diamond, and batch rollback logic is too risky in a contract that cannot be upgraded.

What is the APY?

It floats. Deposit yield depends on how much is lent out; borrowing cost floats between 2% and 46%. The interface shows live values.

What happens if diamond prices drop?

Below the liquidation line, liquidators repay your debt and take part of your collateral. The rest is still yours.

There is no grace period — once the ratio falls below 160%, anyone can liquidate at any moment, with no advance notice. A single liquidation takes only half the debt, but a continuing price decline triggers repeat liquidations; below 105% the whole position is closed at once.

Adding collateral or repaying part of the debt before that happens avoids it.

Will liquidation wipe me out in one go?

Normal liquidation (105%–160%) closes at most 50% of the debt at a time, and the liquidator bonus is **based on the amount repaid**, not on a share of the debt. So liquidating half the debt takes roughly one third of the collateral — after which the ratio jumps above 200% and leaves the danger zone immediately.

But if prices keep falling you will be liquidated again. Below 105% you enter

deep water and the entire debt is closed at once.

Why does redeeming a high-scoring diamond cost more?

To stop people from picking the best diamonds out of the pool. It is a tax on cherry-picking, capped at 30%. Ordinary and low-scoring diamonds pay nothing extra.

Where does protocol revenue go?

The treasury cold wallet `1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`. The address is public and anyone can check its balance.

Can the team run away with the funds?

The contract has no withdrawal function, no admin privileges, and its code is public and auditable. But **the treasury key is a single point** — the only place you have to trust the project. The contract also cannot be upgraded, so do not put everything in.

Where does the diamond price come from?

No external data source. It reads the AMM pool's balance ratio directly, which nobody can forge. A single transaction can move the reference price by at most 10%, making manipulation very expensive.

What happens when a key expires?

Hacash contract state pays rent, and expired keys become inactive (`StorageNotActive`). The contract offers permissionless `rent_keys`: anyone can renew and earn 0.07 HAC per key. This is how the keeper works.

Appendix A: Addresses and Parameters

Item	Value
Deployer	`15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`
Contract nonce	2
Contract address	`T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`
Treasury (cold wallet)	`1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`
hacds serial	**3000** (decimal **0**)
hacds total supply	5,963,298,700,000,000
`SCORE_SCALE`	1e8 (1 point = 100 million hacds)

Item	Value
Total network score	59,632,987 points (= mean 3.55440306 × 16,777,216 diamonds)
`FEE_BPS`	50 (AMM fee 0.5%)
Fee split	LP 80% / treasury 20%
`BORROW_RATIO_BPS`	20,000 (borrow line: 200% collateral ratio, i.e. borrow at most 50%)
`LIQ_COLL_RATIO_BPS`	16,000 (liquidation line: 160%)
`CLOSE_FACTOR_BPS`	5,000 (normal liquidation closes at most 50% of debt)
`DEEP_TRIGGER_BPS`	10,500 (below 105% → deep-water full liquidation)
`LIQ_BONUS_BPS`	500 (5% liquidator bonus, on the amount repaid)
`LIQ_PROTOCOL_BPS`	100 (1% of that to the reserve)
Rate curve	U=0 → 2%, U=35% → 6%, U=50% → 46%
`LENDABLE_AMM_BPS`	3,000 (only 30% of AMM HAC is lendable)
`U_MAX_BPS`	5,000 (global utilization cap 50%)
`BORROWER_CAP_BPS`	2,000 (single borrower cap 20%)
`RES_TARGET_BPS`	300 (bad debt reserve cap 3%)
`MAX_DEPOSIT_COUNT`	200 (per-transaction deposit cap)
`RENT_PERIOD`	300 blocks
`REF_WINDOW`	300 blocks (EMA time constant, about one day)
`EMA_MAX_STEP_BPS`	1,000 (reference price max single move 10%)
`REF_DEV_BPS`	2,000 (circuit breaker: 20% deviation → new borrowing paused)
VM bucket limits	compute 18,009 / resource 6,100 / storage 111,911
Transaction limit	16,384 bytes
Target block time	300 seconds

Appendix B: Units

hacds has **decimal 0**. The integer on-chain is the count itself — 1 means one,

and there are no fractions. This is simpler than the previous edition (decimal 8):

there is no back-and-forth conversion between atoms and "units."

Three rules:

1. **hacds is always counted in units, with no conversion.** Balances, pools, collateral, input fields, quotes — all show the same number.

2. **HAC is a different scale.** The on-chain unit is zhu (1 HAC = 1e8 zhu) and interfaces use HAC throughout. So `add_liquidity(hac_amt, ds_amt)` takes two parameters on different scales: zhu on one side, hacds count on the other. This is the easiest place to make an off-by-1e8 error; we did it ourselves on the first deposit.
3. **The total supply 5,963,298,700,000,000 exceeds 66% of JS `Number.MAX_SAFE_INTEGER`.** Use BigInt in JavaScript and BigInteger in Kotlin. `Long` is not enough.

This document corresponds to the finalized parameters of the contract `HacdsDeFi3000.fitsh`.